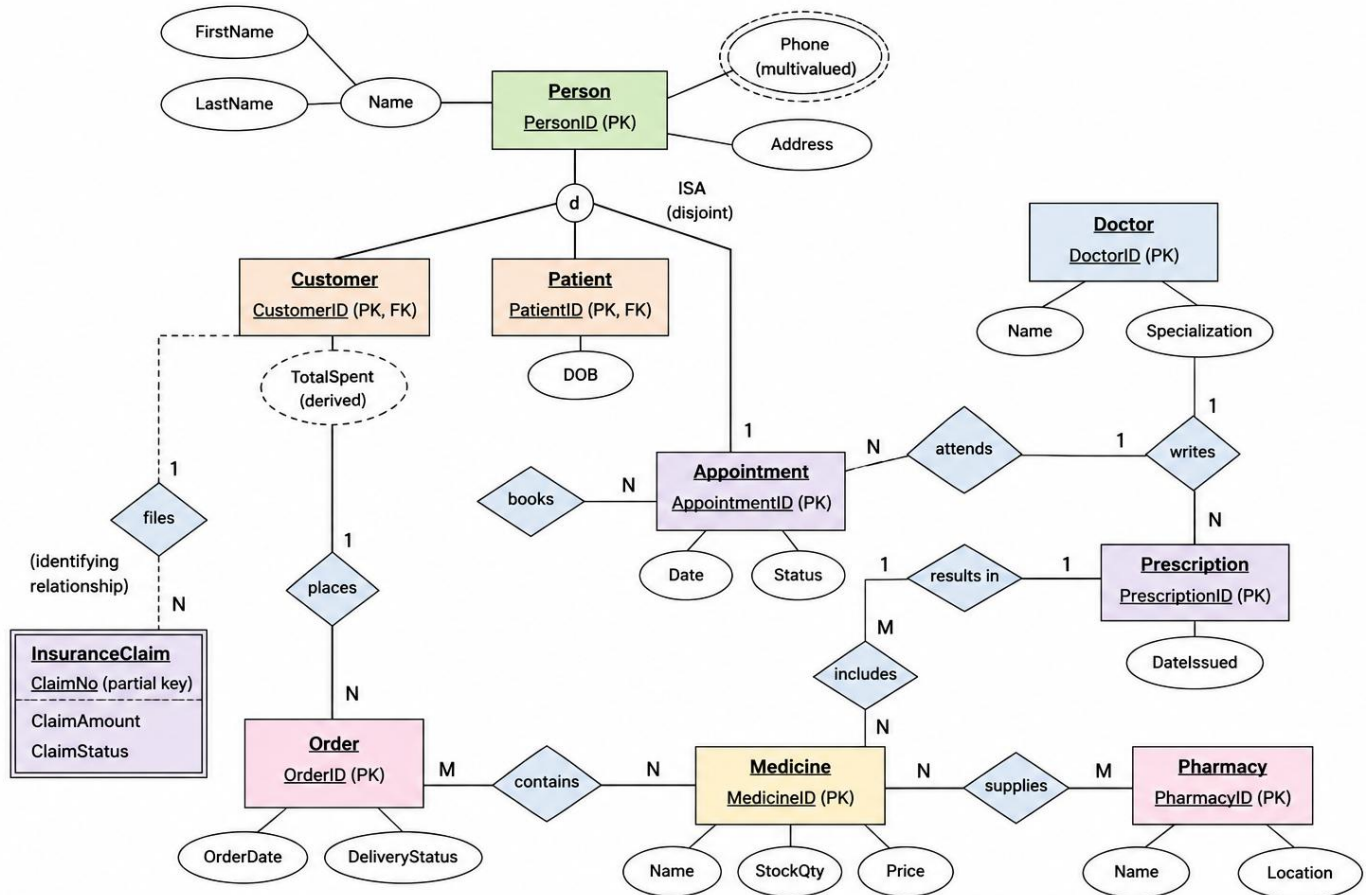


DBMS Assignment 1 (BTS30102)

Integrated E-Commerce & Healthcare Platform — Database Design

Q1. Database Design — E-Commerce & Healthcare Platform

a) EER Diagram — Entities & Relationships (10 Entities)



b) Relational Schema

```
CREATE TABLE Person (  
    PersonID PRIMARY KEY, FirstName, LastName, Address  
);  
  
CREATE TABLE PersonPhone (  
    PersonID, Phone, PRIMARY KEY (PersonID, Phone),  
    FOREIGN KEY (PersonID) REFERENCES Person(PersonID)  
);  
  
CREATE TABLE Customer (  
    CustomerID PRIMARY KEY,  
    FOREIGN KEY (CustomerID) REFERENCES Person(PersonID)  
);  
  
CREATE TABLE Patient (  
    PatientID PRIMARY KEY, DOB,  
    FOREIGN KEY (PatientID) REFERENCES Person(PersonID)  
);  
  
CREATE TABLE Doctor (  
    DoctorID PRIMARY KEY, Name, Specialization  
);  
  
CREATE TABLE Appointment (  
    AppointmentID PRIMARY KEY, PatientID, DoctorID, Date, Status,  
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),  
    FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)  
);
```

```

);
CREATE TABLE Prescription (
    PrescriptionID PRIMARY KEY, AppointmentID UNIQUE, DateIssued,
    FOREIGN KEY (AppointmentID) REFERENCES Appointment(AppointmentID)
);
CREATE TABLE Medicine (
    MedicineID PRIMARY KEY, Name, StockQty, Price
);
CREATE TABLE Pharmacy (
    PharmacyID PRIMARY KEY, Name, Location
);
CREATE TABLE Orders (
    OrderID PRIMARY KEY, CustomerID, OrderDate, DeliveryStatus,
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);

CREATE TABLE OrderItem (
    OrderID, MedicineID, Quantity,
    PRIMARY KEY (OrderID, MedicineID),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (MedicineID) REFERENCES Medicine(MedicineID)
);
CREATE TABLE PrescriptionItem (
    PrescriptionID, MedicineID,
    PRIMARY KEY (PrescriptionID, MedicineID),
    FOREIGN KEY (PrescriptionID) REFERENCES Prescription(PrescriptionID),
    FOREIGN KEY (MedicineID) REFERENCES Medicine(MedicineID)
);
CREATE TABLE Supplies (
    PharmacyID, MedicineID,
    PRIMARY KEY (PharmacyID, MedicineID),
    FOREIGN KEY (PharmacyID) REFERENCES Pharmacy(PharmacyID),
    FOREIGN KEY (MedicineID) REFERENCES Medicine(MedicineID)
);
CREATE TABLE InsuranceClaim (
    PatientID, ClaimNo, ClaimAmount, ClaimStatus,
    PRIMARY KEY (PatientID, ClaimNo),
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)
);
);

```

c) Functional Dependencies

Consider the un-normalized relation used for order processing:

OrderDetails (OrderID, MedicineID, CustomerID, CustomerName, MName, Quantity, Price)

Candidate key: {OrderID, MedicineID}

- OrderID → CustomerID, CustomerName — partial dependency (depends on part of the key)
- MedicineID → MName, Price — partial dependency
- CustomerID → CustomerName — transitive dependency (OrderID → CustomerID → CustomerName)
- {OrderID, MedicineID} → Quantity — full functional dependency (no partial/transitive issue)

d) Normalization (1NF → 3NF)

- 1NF: All attributes above are already atomic (single-valued), so OrderDetails satisfies 1NF.
- 2NF: Remove partial dependencies → split into Orders(OrderID, CustomerID, CustomerName), Medicine(MedicineID, MName, Price) and OrderItem(OrderID, MedicineID, Quantity).
- 3NF: Remove transitive dependency CustomerID → CustomerName from Orders → split into Orders(OrderID, CustomerID) and Customer(CustomerID, CustomerName).

This removes insertion anomalies (a medicine's price can be added without an order), deletion anomalies (deleting the last order of a customer no longer deletes the customer's name), and update anomalies (customer name / medicine price is now updated in one place only).

Q2. Relational Algebra Expressions

(σ = select, π = project, $\bowtie$ = join, γ = group-by/aggregate, using the schema from Q1)

i) Find high-value customers.

$\sigma_{\text{TotalSpent} > 50000}(\gamma_{\text{CustomerID}}, \text{SUM}(\text{Quantity} \times \text{Price}) \rightarrow \text{TotalSpent}(\text{Orders} \bowtie \text{OrderItem} \bowtie \text{Medicine}))$

ii) Find delayed orders.

$\sigma_{\text{DeliveryStatus} = \text{'Delayed'}}(\text{Orders})$

iii) Find medicines with low or unavailable stock.

$\sigma_{\text{StockQty} < 10}(\text{Medicine})$

iv) Find rejected insurance claims.

$\sigma_{\text{ClaimStatus} = \text{'Rejected'}}(\text{InsuranceClaim})$

v) Find doctors with a high number of patient visits.

$\pi_{\text{DoctorID}}(\sigma_{\text{VisitCount} > 50}(\gamma_{\text{DoctorID}}, \text{COUNT}(\text{PatientID}) \rightarrow \text{VisitCount}(\text{Appointment})))$

vi) Find pharmacies supplying a particular medicine.

$\pi_{\text{Pharmacy.Name}}(\text{Pharmacy} \bowtie \text{Supplies} \bowtie \sigma_{\text{Medicine.Name} = \text{'Paracetamol'}}(\text{Medicine}))$